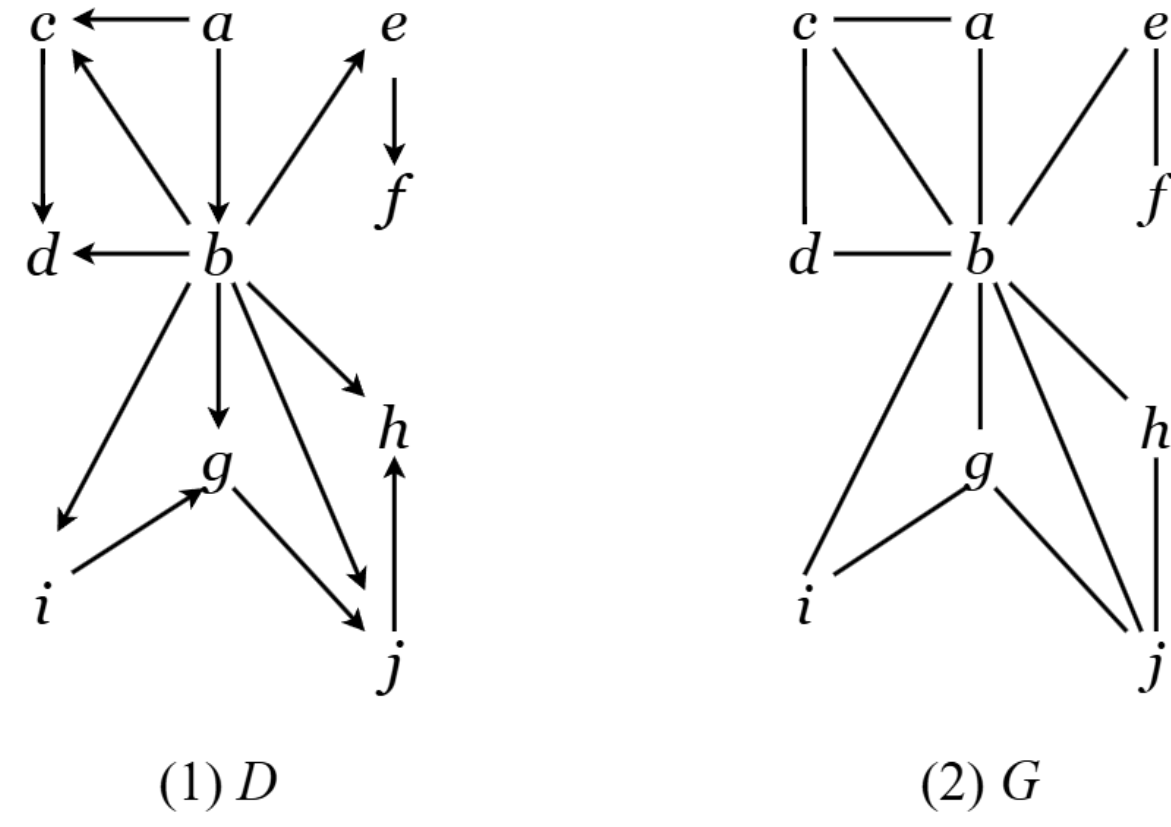




Introduction

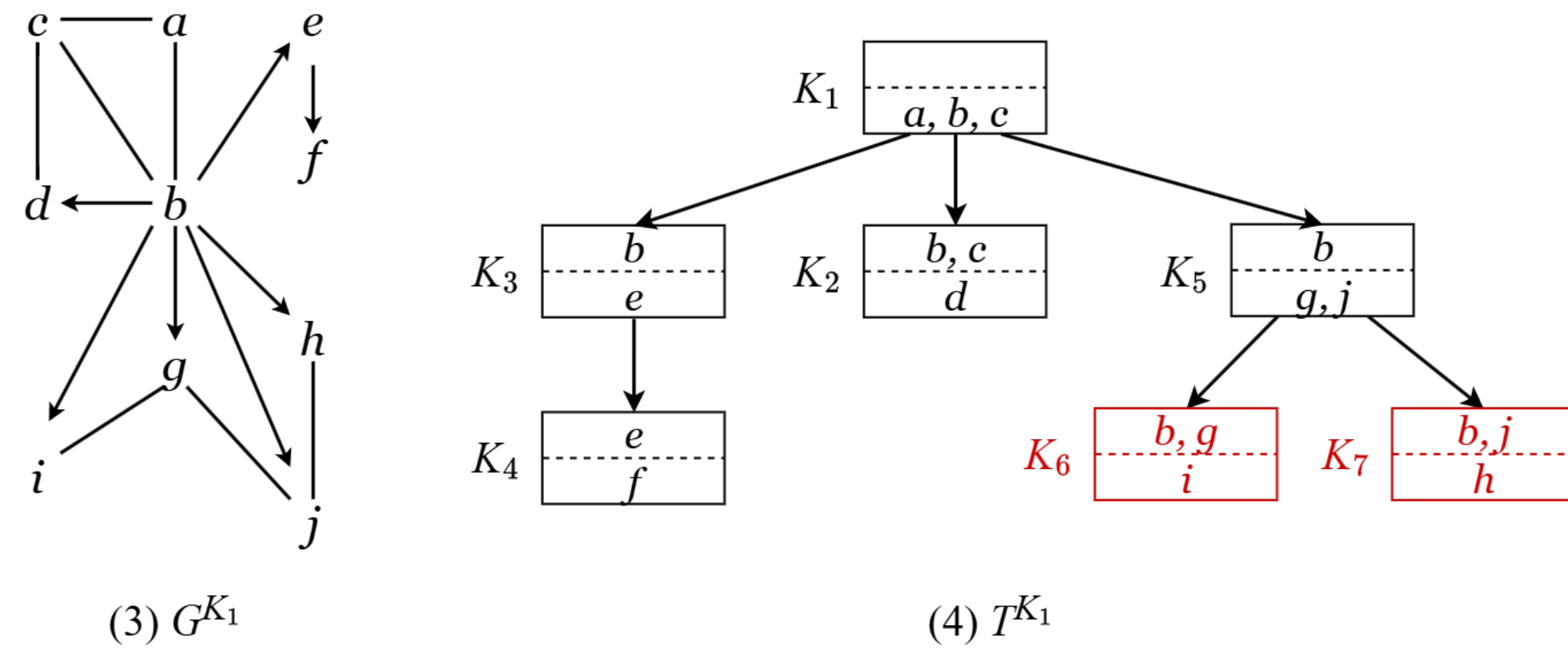
Directed acyclic graphs (DAGs) are widely used to represent multivariate causal structures across diverse fields, includ ing epidemiology, biology, and economics.



- Causal structure can not be uniquely determined from observational data alone.
- Markov equivalence class (MEC) encode the same conditional independencies.
- The size of an MEC, defined as the number of DAGs within the class, plays a critical role in the design of causal intervention experiments (He & Geng, 2008) and average causal effect estimation (Maathuis et al., 2009).
- Yangbo He, Jinzhu Jia, and Bin Yu (2015) first introduced the Root-Picking algorithm.

Clique-Picking Algorithm

Wienöbst et al. (2023) introduces the **Clique-Picking** (CP) algorithm, which is a polynomial-time algorithm for determining the size of a MEC. For a given chordal graph $G = (V, E)$, the algorithm needs to recursively select a clique as root, introduce outgoing edges from the root, and determine smaller undirected connected components (UCCG) from the resulting graph.



- Maximal cliques: $K_1 = \{a, b, c\}$, $K_2 = \{b, c, d\}$, $K_3 = \{b, e\}$, $K_4 = \{e, f\}$, $K_5 = \{b, g, j\}$, $K_6 = \{b, g, i\}$ and $K_7 = \{b, h, j\}$.
- For G^{K_1} , we have the UCCGs: $C_G(K_1) = \{G[e], G[d], G[f], G[g, h, i, j]\}$.
- The iterative formula

$$\text{Size}(G) = \sum_{p=1}^m \phi(K_p, \text{FP}(K_p, T^{K_1})) \cdot \prod_{J \in C_G(K_p)} \text{Size}(J), \quad (1)$$

Suppose there are m maximal cliques in G , this intensive process has a cost of $O(m(|V| + |E|))$. We propose the **Super Cliques Transfer** algorithm, which runs in time $O(m^2)$.

High-level Structures

Given a UCCG G , a clique tree T^{K_1} with some root clique K_1 can be generated via the MCS algorithm.

- For each clique K_p , its **separator** S_p is as $S_p = K_p \cap (K_1 \cup \dots \cup K_{p-1}) = K_p \cap K_i$;
- its **residual** R_p is defined as $R_p = K_p \setminus S_p$.
- For the root clique, we simply set $S_1 = \emptyset$ and $R_1 = K_1$.

(Clique header, clique tail) Let T^{K_1} be a rooted clique tree with RIP clique order $K_1, K_2, \dots, K_m$.

- For any $p = 2, \dots, m$, K_p is a *clique header* within T^{K_1} if for any ancestral clique K_q of K_p with $q \neq 1$, the corresponding S_q is not a proper subset of S_p .
- For $p = 2, \dots, m$, suppose K_p is a clique header within T^{K_1} , a descendant clique K_q of K_p is a *clique tail* that follows K_p if $S_p \subsetneq S_q$.

(Super clique, super residual) Within a clique tree T^{K_1} , suppose K_p is a clique header and $K_{p_1}, \dots, K_{p_r}$ are all its clique tails.

- The clique set $\text{SK}_{p^+}^{(1)} = \text{SK}_{p|p_1, \dots, p_r}^{(1)} := \{K_p, K_{p_1}, \dots, K_{p_r}\}$ is called a *super clique*.
- The set of the residuals corresponding to the cliques within $\text{SK}_{p^+}^{(1)}$ is called a *super residual*, and denoted as

$$\text{SR}_{p^+}^{(1)} = \text{SR}_{p|p_1, \dots, p_r}^{(1)} := \{R_p, R_{p_1}, \dots, R_{p_r}\}.$$

In the example, K_2, K_3, K_4 and K_5 are clique headers within T^{K_1} , K_6 and K_7 are the clique tails that follows K_5 . There are four super cliques: $\text{SK}_2^{(1)} = \{K_2\}$, $\text{SK}_3^{(1)} = \{K_3\}$, $\text{SK}_4^{(1)} = \{K_4\}$ and $\text{SK}_{5|6,7}^{(1)} = \{K_5, K_6, K_7\}$ within T^{K_1} . We can see $G[\text{SR}_2^{(1)}] = G[d]$, $G[\text{SR}_3^{(1)}] = G[e]$, $G[\text{SR}_4^{(1)}] = G[f]$, and $G[\text{SR}_{5|6,7}^{(1)}] = G[g, h, i, j]$ are the undirected connected components in $C_G(K_1)$.

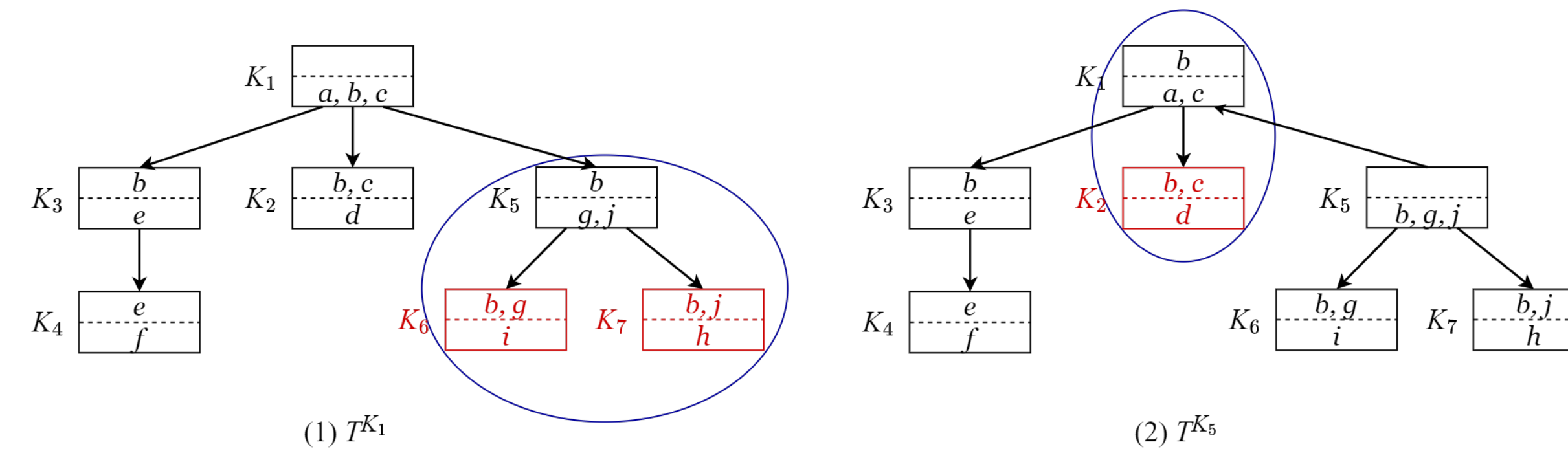
Theorem 1

Let T^{K_1} be a rooted clique tree of a chordal graph G with MCS clique order $K_1, K_2, \dots, K_m$. Then

$$C_G(K_1) = \left\{ G[\text{SR}_{p^+}^{(1)}] : \text{SR}_{p^+}^{(1)} \text{ is a super residual within } T^{K_1} \right\}.$$

For a given T^{K_1} , super cliques create operation (**SC-Create-Op**) is designed to return the set $L^{(1)}$ of all super cliques, all separators and residuals, and $C_G(K_1)$.

The Super Cliques Transfer Operation



Suppose K_i is one child clique of K_1 in T^{K_1} , we can easily identify all super cliques and super residuals of T^{K_i} from those of T^{K_1} . This leads to the super clique transfer operation (**SC-Trans-Op**), which derives the set of super cliques of T^{K_i} from that of T^{K_1} , and generates $C_G(K_i)$ from $C_G(K_1)$.

- Consider the super clique $\text{SK}_{i^+}^{(1)}$ with clique header K_i in T^{K_1} . Suppose K_i has h child clique(s) in T^{K_1} : $K_{p_1}, \dots, K_{p_h}$. Then, the super clique $\text{SK}_{i^+}^{(1)}$ of T^{K_1} get split into h super clique(s) in T^{K_i} : $\text{SK}_{p_1^+}^{(i)}, \dots, \text{SK}_{p_h^+}^{(i)}$. These super cliques have $K_{p_1}, \dots, K_{p_h}$ as their clique headers, respectively.
- Consider all child cliques of K_1 in T^{K_1} but with K_i excluded. Among these child cliques, select those that become clique tails of K_1 in T^{K_i} , and denote these selected cliques as $K_{p_1}, \dots, K_{p_h}$. Then, their corresponding super cliques $\text{SK}_{p_1^+}^{(1)}, \dots, \text{SK}_{p_h^+}^{(1)}$ in T^{K_1} , together with K_1 , will form a new super clique in T^{K_i} :

$$\text{SK}_{1^+}^{(i)} = \{K_1\} \cup \text{SK}_{p_1^+}^{(1)} \cup \dots \cup \text{SK}_{p_h^+}^{(1)}. \quad (2)$$

Note, if there does not exists any clique tail of K_1 in T^{K_i} , then (2) simply becomes $\text{SK}_{1^+}^{(i)} = \{K_1\}$.

- For the child cliques of K_1 in T^{K_1} that are *not* selected for (2), their corresponding super cliques remain unchanged and continue to constitute super cliques in T^{K_i} . All other super cliques in T^{K_1} remain unchanged and continue to form super cliques in T^{K_i} .

Experiments

We now evaluate our proposed SC-Trans algorithm. It is integrated into the Clique-Picking (**CP**) algorithm. The improved version is denoted as **ICP**. We compare the practical performance of our improved ICP and the state-of-the-art CP algorithm in a series of experiments.

We performed experiments with $|V| = 512, 1024, 2048, 4096$. The **graph density** r is measured by $|E|/|E_{\max}|$ with $|E_{\max}| = |V|(|V| - 1)/2$. We chose r as either $\mathbf{r} = 0.34$ or $\mathbf{r} = 0.08$ and generated ten chordal graphs for each number of graph vertices. Let T_{CP} denote the average running time of CP algorithm, and let T_{ICP} denote the average running time of our ICP algorithm.

Table 1. Average running time (in seconds)

r	0.34		0.08	
$ V $	T_{ICP}	T_{CP}	T_{ICP}	T_{CP}
512	1.022	2.081	0.842	1.494
1024	8.048	15.579	5.586	9.974
2048	68.048	125.181	40.545	74.940
4096	707.484	1169.811	387.491	660.435

Performance comparison for varying graph density r and the ratio $\theta = m/(|V| + |E|)$. Step 2 of the clique-picking algorithm is also isolated for comparison.

Table 2. Comparison between ICP and CP (the runtime difference $T_{\text{CP}} - T_{\text{ICP}}$ and $T_{\text{CP},2} - T_{\text{ICP},2}$ are measured in seconds)

$ V $		1024			2048			
r	θ	$T_{\text{CP}} - T_{\text{ICP}}$	$T_{\text{CP},2} - T_{\text{ICP},2}$	$T_{\text{CP},2}/T_{\text{ICP},2}$	θ	$T_{\text{CP}} - T_{\text{ICP}}$	$T_{\text{CP},2} - T_{\text{ICP},2}$	$T_{\text{CP},2}/T_{\text{ICP},2}$
0.04	0.0381	2.17	2.33	5.34	0.0189	20.45	17.99	12.27
0.06	0.0243	2.84	3.30	7.56	0.0126	27.87	24.19	17.16
0.12	0.0108	5.01	4.73	18.43	0.0054	45.58	37.41	39.70
0.24	0.0044	7.02	7.35	47.59	0.0022	60.40	76.63	95.35
0.46	0.0015	8.42	9.20	153.55	0.0007	80.97	81.45	264.54